

Integration Architecture Guidelines

Asteria Digital Services | IAG-INT-002 | Version 2.1 | Effective 2026-06-15

Owner: Integration Architecture

Synthetic training document - created for the Week 2 RAG project. No real company policy is represented.

1. Purpose

These guidelines define preferred integration patterns for connecting applications, SaaS platforms, data services, and asynchronous messaging components.

2. Choosing Synchronous vs Asynchronous Integration

Pattern	Use When
Synchronous	Use when the caller requires an immediate business response and the downstream dependency can meet the end-to-end latency target.
Asynchronous	Prefer when processing is long-running, can be decoupled, must absorb bursts, or must continue despite temporary downstream unavailability.
Batch	Use for scheduled high-volume transfer when near-real-time processing is not required.
Event-driven	Use when a business event should notify multiple consumers without tight coupling to the producer.

3. Timeouts and Retries

- Every synchronous downstream call **MUST** have a finite connection and response timeout.
- Transient integration failures **MAY** be retried up to 3 attempts total (initial attempt plus 2 retries) unless a more restrictive downstream contract exists.
- Retries **SHOULD** use exponential backoff with jitter.
- Functional errors, validation errors, authentication failures, and authorization failures **MUST NOT** be blindly retried.
- A retryable operation that can create duplicates **MUST** be idempotent or protected by an idempotency key.

4. Messaging and Delivery

- Business messages **SHOULD** include messageId, correlationId, eventType, schemaVersion, and occurredAt.

- Consumers **MUST** be designed for at-least-once delivery unless the platform and business process provide a stronger guarantee.
- Consumers **SHOULD** detect duplicate messageId values when duplicate processing would cause incorrect business outcomes.
- After configured retries are exhausted, technically failed messages **SHOULD** be moved to a dead-letter queue (DLQ) or equivalent error channel.
- Messages with permanent data or functional errors **SHOULD** not loop indefinitely through automated retries.

5. Correlation and Traceability

The correlationId received from an API or upstream message **SHOULD** be propagated unchanged across the integration path. If no correlationId is available, the first integration component **MUST** create one. Logging must comply with LMS-OPS-003.

6. Contract and Schema Management

- Interfaces **SHOULD** use explicit schemas and versioned contracts.
- Breaking schema changes require a new major schema or API version.
- Consumers **SHOULD** ignore unknown optional fields unless the contract explicitly forbids them.
- Canonical models may be used when they reduce repeated mappings across multiple systems; point-to-point pass-through is acceptable when no transformation or reuse benefit exists.

7. Failure Classification

Failure Type	Expected Handling
Technical	Timeouts, temporary service unavailable, transient network failures. Retry may be appropriate.
Functional	Business rule rejection or invalid state transition. Route for business correction; do not automatically retry.
Data	Malformed or incomplete input. Quarantine or route for correction; do not automatically retry.
Security	Authentication or authorization failure. Stop processing and raise a security/operational alert as appropriate.

8. Related Standards

- API Design Standards - ADS-API-STD-001
- Logging & Monitoring Standard - LMS-OPS-003
- Architecture Principles - EAP-ARCH-001